

*Máster en Inteligencia Artificial*  
*para Liderazgo Técnico y Transformación Empresarial*

---

# Seguridad en el Ciclo de Desarrollo de IA

## Módulo 12 · Seguridad en IA

---

Secure AI DLC, Shift-Left, STRIDE para IA, CI/CD de seguridad, Security Champion y recursos OWASP/NIST/Google SAIF.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales  
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

## Seguridad en el Ciclo de Desarrollo de IA (AI SecDevOps)

El enfoque tradicional de la seguridad de software era "security as an afterthought": los equipos desarrollaban el sistema y un equipo de seguridad lo revisaba antes de lanzarlo. Este enfoque es especialmente inadecuado para sistemas de IA: los vectores de ataque son más complejos, el comportamiento es más difícil de predecir, y las superficies de ataque aparecen en múltiples capas (datos, modelos, prompts, herramientas, infraestructura). El enfoque correcto es integrar la seguridad en cada fase del ciclo de desarrollo: Shift-Left Security para sistemas de IA.

El EU AI Act (artículo 15) requiere que los sistemas de IA de alto riesgo implementen robustez y ciberseguridad a lo largo de todo el ciclo de vida. El Executive Order 14110 de EE.UU. (en vigor desde octubre 2023) requiere que los desarrolladores de modelos frontier entreguen los resultados de red teaming pre-lanzamiento a las agencias federales. La seguridad ya no es opcional ni post-hoc para sistemas de IA en producción.

## Secure AI Development Lifecycle (Secure AI DLC)

### Fase 1: Diseño seguro

En la fase de diseño, la seguridad se incorpora como requisito desde el principio: threat modeling (¿cuáles son los vectores de ataque posibles para este sistema?), análisis de riesgo por componente (¿qué daño puede causar cada componente si es comprometido?), diseño del modelo de permisos y confianza (¿qué puede hacer cada componente?, ¿qué confianza tiene?), y definición de los requisitos de seguridad no negociables (autenticación, logging, rate limiting). El STRIDE model (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) aplicado a sistemas de IA produce un threat model comprensivo.

### Fase 2: Desarrollo seguro

En la fase de desarrollo, las prácticas de Secure Coding se extienden a los componentes específicos de IA: los prompts se tratan como código (en control de versiones, con tests), los datasets se verifican en origen y se versionan con DVC, las dependencias de modelos se verifican con hashes y se escanean con ModelScan, y el código de herramientas del agente tiene validación de parámetros. Los code reviews incluyen reviewers específicos para cada aspecto de seguridad de IA.

### Fase 3: Testing de seguridad

El testing de seguridad para sistemas de IA incluye: SAST para el código Python del sistema (Bandit, Semgrep), tests de las herramientas del agente con inputs maliciosos (validación de que los parámetros fuera de rango son rechazados), red teaming del LLM con Garak y DeepTeam sobre el golden dataset de seguridad (el conjunto de prompts adversariales que el sistema debe rechazar), testing de límites de rate y consumo, y testing de aislamiento de tenants (verificar que los datos de un usuario no son accesibles para otro).

---

## Integración de seguridad en CI/CD para IA

El pipeline de CI/CD para un sistema de IA con Shift-Left Security: Pre-commit hooks: detectar secretos en el código (detect-secrets, git-secrets), linting de seguridad (Bandit para Python). Pull Request checks: SAST con Semgrep, escaneo de dependencias (pip-audit), ModelScan sobre nuevos modelos añadidos al repositorio. Build pipeline: test suite de seguridad (tests de validación de herramientas, tests de filtros de input/output), tests de rate limiting, tests de aislamiento de tenants. Pre-deploy: red teaming automatizado con Garak sobre el sistema completo, generación del informe de seguridad que debe aprobar el Security Champion antes del despliegue.

El concepto de Security Champion: designar un miembro del equipo de desarrollo como Security Champion para el proyecto de IA. El Security Champion no es el equipo de seguridad centralizado: es alguien del equipo de desarrollo con formación adicional en seguridad de IA que actúa como punto de contacto, promueve buenas prácticas, y hace la primera revisión de seguridad de los PRs. Este modelo (adoptado por empresas como Spotify y Netflix para software general) se está adaptando para equipos de IA.

### Gestión de la deuda técnica de seguridad

Los sistemas de IA acumulan deuda técnica de seguridad rápidamente: prompts que nunca se actualizan aunque el modelo haya cambiado de comportamiento, dependencias de librerías con vulnerabilidades conocidas que no se han actualizado, y coverage de tests de seguridad que no cubre los nuevos tipos de ataque que aparecen cada mes. Define un proceso explícito de gestión de deuda técnica de seguridad: revisión mensual de las dependencias, actualización trimestral del golden dataset adversarial, y revisión semestral completa del threat model del sistema.

---

#### SECCIÓN 4 · EJEMPLOS REALES

- Microsoft Secure AI Development Lifecycle: Microsoft ha publicado su AI Secure Development Lifecycle que extiende su SDL clásico con componentes específicos de IA: threat modeling para sistemas generativos, red teaming con PyRIT en CI/CD, gestión de contenido y seguridad en Azure OpenAI, y gobernanza de modelos con el Model Registry de Azure ML. La integración de PyRIT (Python Risk Identification Toolkit) en los pipelines de CI/CD es el componente más innovador.
- Checklist de seguridad pre-lanzamiento de Anthropic (publicado en la documentación de Anthropic): Anthropic publica su proceso de evaluación de seguridad pre-lanzamiento para los modelos Claude, que incluye: evaluaciones de capacidades peligrosas (CBRN), tests de jailbreak exhaustivos con el red team interno, evaluación de uso dual, y publicación de un model card con los resultados de seguridad. Este proceso se ha convertido en referencia para la industria.

---

#### SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

**Error 1: Separar completamente el equipo de seguridad del equipo de desarrollo de IA. Los expertos en seguridad de IA y los desarrolladores de sistemas de IA necesitan trabajar de forma integrada: los atacantes no esperan a la reunión de security review. El**

modelo de Security Champion (embedded security expertise en el equipo de desarrollo) es más efectivo que el modelo de security gate (revisión externa antes del lanzamiento).

Error 2: No actualizar el threat model cuando el sistema cambia. El threat model del día del lanzamiento puede quedar obsoleto en semanas si el sistema añade nuevas herramientas, nuevos tipos de usuarios, o se integra con nuevos sistemas. Revisa el threat model con cada release mayor del sistema.

Error 3: Tratar los resultados del red teaming como secretos internos. Compartir los hallazgos de seguridad internamente (en el equipo de desarrollo, en el equipo de producto, en la dirección) produce mejores resultados que mantenerlos en silos. La transparencia interna sobre los riesgos de seguridad identificados produce una cultura de mejora continua.

## SECCIÓN 6 · APLICACIÓN PRÁCTICA

---

El minimal viable Secure AI DLC: define el threat model del sistema (una hora de análisis estructurado con el equipo). Crea un golden dataset adversarial (20-50 prompts que el sistema debe rechazar definitivamente). Integra las pruebas del golden dataset adversarial en el CI. Implementa Garak en el CI para el testing de seguridad automatizado. Designa un Security Champion del equipo. Define el proceso de respuesta a incidentes de seguridad (quien es el responsable, cómo se escala, cómo se comunica). Con estas seis piezas, tienes un Secure AI DLC básico que puede evolucionar.

Para el threat modeling de un sistema LLM: usa el STRIDE model adaptado a IA. Spoofing: ¿puede un usuario hacerse pasar por otro usuario o por el sistema? Tampering: ¿puede modificarse el corpus RAG o los pesos del modelo maliciosamente? Repudiation: ¿podemos demostrar qué acciones tomó el sistema y cuándo? Information Disclosure: ¿puede el sistema revelar datos de otros usuarios o el system prompt? Denial of Service: ¿puede saturarse el sistema con inputs masivos? Elevation of Privilege: ¿puede un usuario no autorizado obtener capacidades que no debería tener?

## SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

---

- M8-T9 (CI/CD para IA): el Secure AI DLC se integra en el pipeline de CI/CD estándar.
- M12-T9 (Red teaming): el red teaming es el componente de testing de seguridad del Secure AI DLC.
- M13 (Gobernanza): el Secure AI DLC produce la documentación de seguridad requerida por el AI Act.

## SECCIÓN 8 · RESUMEN EJECUTIVO

---

El Secure AI Development Lifecycle (Secure AI DLC) integra la seguridad en cada fase: diseño (threat modeling con STRIDE), desarrollo (prompts como código, DVC para datasets, ModelScan para dependencias), testing (SAST, red teaming automatizado con Garak, tests de herramientas). El pipeline CI/CD incluye: pre-commit (detect-secrets), PR checks (Bandit, pip-audit, ModelScan), y pre-deploy (red teaming, informe de seguridad para el Security Champion). El minimal viable Secure AI DLC: threat model + golden dataset adversarial + CI tests + Garak + Security Champion + proceso de incidentes.

Recursos: OWASP Gen AI Red Teaming Guide, NIST AI RMF, Google SAIF.